# Appunti del corso di
# Basi di dati

UniVR - Dipartimento di Informatica

secondo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

# Indice


**1 Tecnologie per le basi di dati** **1**
1.1 Tecnologia dei DBMS . . . 1
1.1.1 Transazione . . . 2
1.1.2 Atomicità . . . 3
1.1.2.1 Consistenza . . . 3
1.1.2.2 Isolamento . . . 4
1.1.2.3 Persistenza . . . 4
1.2 Architettura di un DBMS . . . 4

**2 SQL** **7**
2.1 Istruzione SELECT . . . 8
2.1.1 Struttura . . . 8
2.1.2 Interrogazione: Clausola SELECT . . . 9
2.1.3 Interrogazione: Clausola SELECT ALL . . . 9
2.1.4 Interrogazione: Clasuola FROM . . . 9
2.1.5 Interrogazione: Clausola WHERE . . . 10
2.2 Clausola WHERE . . . 12
2.2.1 Operatore LIKE . . . 12
2.2.2 Operatore SIMILAR TO . . . 12
2.2.3 Operatore BETWEEN . . . 13
2.2.4 Operatore IN . . . 13
2.2.5 Operatore IS NULL . . . 14
2.3 SQL e ALGEBRA RELAZIONALE . . . 15
2.3.1 Selezione e proiezione . . . 15
2.3.2 SELECT: Senza proiezione . . . 16
2.3.3 Proiezione: gestione dei duplicati in SQL . . . 16
2.3.4 SELECT: Selezione, Proiezione, Join . . . 17
2.3.5 Interrogazioni con JOIN . . . 17
2.3.5.1 Interrogazioni con INNER JOIN . . . 18
2.3.5.2 Interrogazioni con LEFT OUTER JOIN . . . 19

# 1

# Tecnologie per le basi di dati

Argomenti visti nel Modulo di Teoria:

- Progettazione concettuale
- Progettazione logica
- Linguaggi di interrogazione (algebra relazionale e calcolo relazionale)

## 1.1 Tecnologia dei DBMS

Un DBMS **(Database Management System)** o sistema di gestione di basi di dati è un sistema software in grado di gestire collezioni di dati che siano e **persistenti** assicurando la loro **affidabilità** e **privatezza**.

Un DBMS in quanto sistema software deve essere efficiente ed efficace e deve inoltre estendere le funzionalità del file system. Esempio: Gestione in memoria secondaria collezioni di dati.

**Definizione: Basi di dati**

Una base di dati è una collezione di dati gestita da un DBMS.

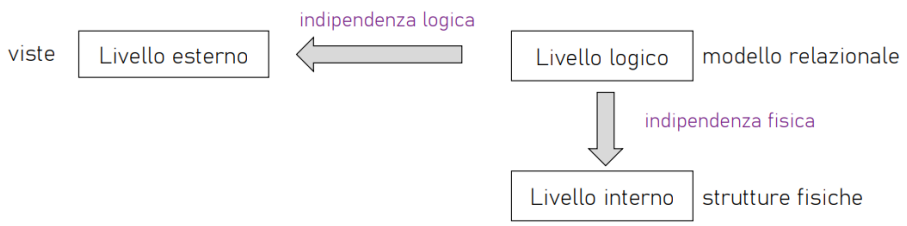



**Figura 1.1:** Architettura su 3 livelli del DBMS.

Un DBMS basato sul modello relazionale è nella maggior parte dei casi anche un sistema transazionale: fornisce un meccanismo per la definizione ed esecuzione di transazioni.

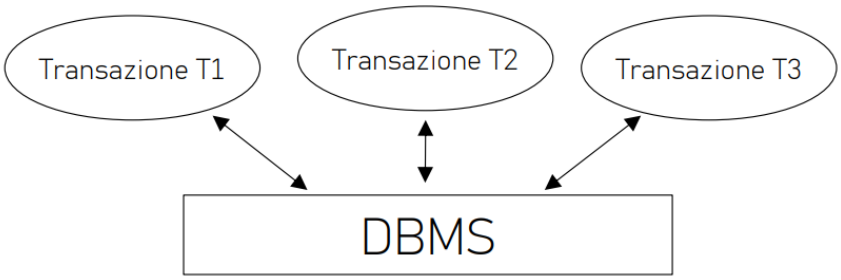



**Figura 1.2:** Ssistema transizionale

In sintesi, durante il corso si affronteranno i seguenti argomenti:

- Transazioni: definizione e proprietà
- Architettura di un DBMS:
  - Gestore delle interrogazioni: scrittura query in SQL
  - Gestore delle interrogazioni: ottimizzazione delle interrogazioni
  - Gestore dei metodi di accesso: indici primari e secondari, indici B+-tree e indici Hash
  - Gestore del buffer: interazione tra memoria secondaria e memoria centrale
  - Gestore della memoria secondaria: strutture fisiche per la memorizzazione dei dati in memoria secondari
  - Gestore della concorrenza: gestione dell'esecuzione concorrente (VSR, CSR, locking a due fasi)
  - Gestore dell'affidabilità: gestione dei guasti
- Architettura di un sistema document-based

### 1.1.1 Transazione

**Definizione: Transazione**

È un'unità di lavoro svolta da un programma applicativo (che interagisce con una base di dati) per la quale si vogliono garantire proprietà di correttezza, robustezza e isolamento.

Le caratteristiche principali sono:

- Una transazione o va a buon fine e ha effetto sulla base di dati o abortisce e non ha nessun effetto sulla base di dati.
- O tutto o niente! Non sono ammesse esecuzioni parziali

Sintassi per definire una transazione in SQL:

```
<transazione> → begin transaction
                <programma>
                end transaction
<programma> → ⟨ <istruzione> | commit work | rollback work ⟩
              { ⟨ <istruzione> | commit work | rollback work ⟩}
```

La transazione va a buon fine all'esecuzione di un commit work ma non ha alcun effetto se viene eseguito un rollback work.

Una transazione è ben formata se:

- Inizia con un begin transaction.
- Termina con un end transaction.
- La sua esecuzione comporta il raggiungimento di un commit o di un rollback work e dopo il commit/rollback non si eseguono altri accessi alla base di dati.

Esempio di transazione ben formata:

```
begin transaction
    update CONTO set saldo = saldo - 1200
        where filiale = '005' and numero = 15;
    update CONTO set saldo = saldo + 1200
        where filiale = '005' and numero = 205
    commit work;
end transaction
```

**Proprietà delle transazioni**

Una transazione ha quattro proprietà:
- Atomicità
- Consistenza
- Isolamento
- Persistenza

### 1.1.2 Atomicità

Una transazione è una unità di esecuzione **indivisibile**. O viene eseguita completamente o non viene eseguita affatto.

Implicazioni:

- Se una transazione viene **interrotta prima** del commit, il lavoro fin qui eseguito dalla transazione deve essere disfatto ripristinando la situazione in cui si trovava la base di dati prima dell'inizio della transazione.
- Se una transazione viene **interrotta dopo** l'esecuzione del commit (commit eseguito con successo), il sistema deve assicurare che la transazione abbia effetto sulla base di dati.

#### 1.1.2.1 Consistenza

L'esecuzione di una transazione non deve violare i **vincoli di integrità**.

Implicazioni:

**Verifica immediata** :
- Fatta nel corso della transazione
- Viene abortita solo l'ultima operazione e il sistema restituisce all'applicazione una segnalazione d'errore
- L'applicazione può quindi reagire alla violazione.

**Verifica differita** :

- Al commit se un vincolo di integrità viene violato la transazione viene abortita senza possibilità da parte dell'applicazione di reagire alla violazione

#### 1.1.2.2 Isolamento

L'esecuzione di una transazione deve essere **indipendente** dalla contemporanea esecuzione di altre transazioni.

Implicazioni:

- Il rollback di una transazione non deve creare rollback a catena di altre transazioni che si trovano in esecuzione contemporaneamente.
- Il sistema deve regolare l'esecuzione concorrente con meccanismi di controllo dell'accesso alle risorse

#### 1.1.2.3 Persistenza

L'effetto di una transazione che ha eseguito il commit non deve andare perso. Implicazioni:

- Il sistema deve essere in grado, in caso di guasto, di garantire gli effetti delle transazioni che al momento del guasto avevano già eseguito un commit.

## 1.2 Architettura di un DBMS

L'architettura mostra i moduli principali che possiamo individuare nei DBMS attuali, considerando le diverse funzionalità che il DBMS svolge durante l'esecuzione delle transazioni. Per ogni modulo dell'architettura presentiamo le funzionalità che esso svolge e alcune delle tecniche che applicativo

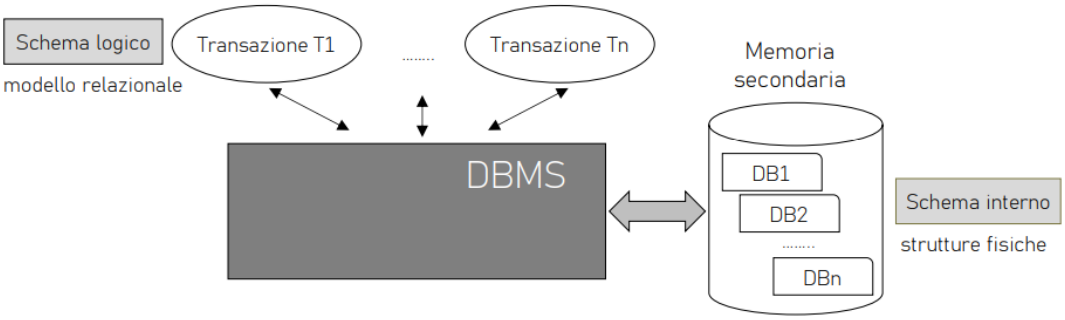



**Figura 1.3:** Architettura di riferimento di un DBMS.

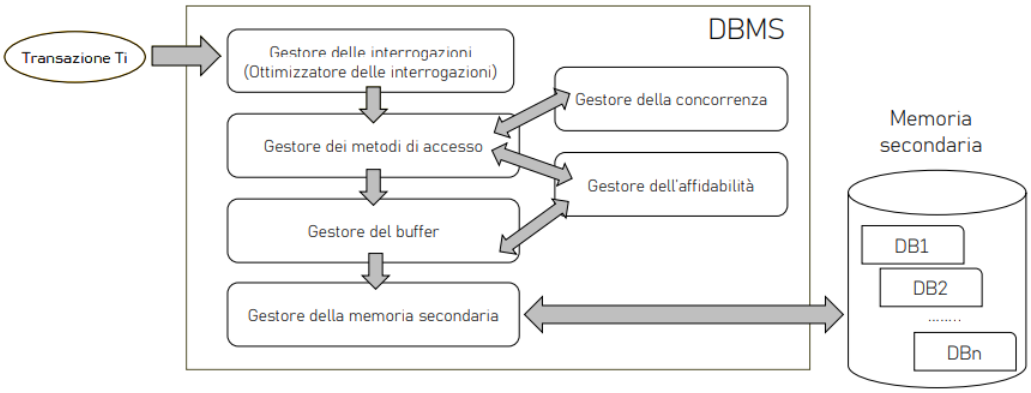
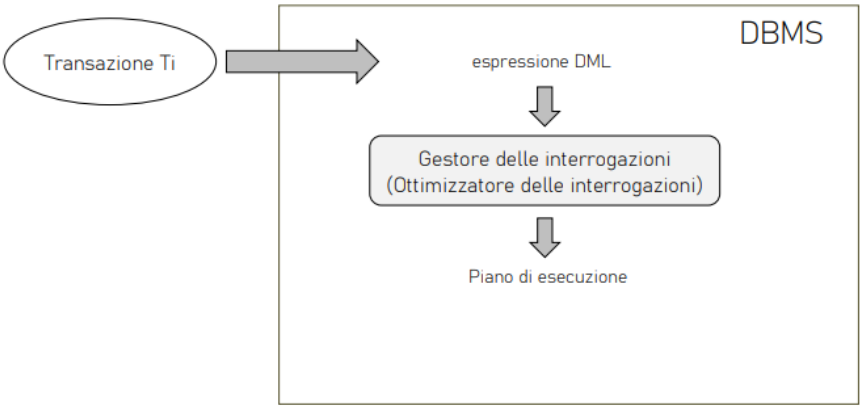
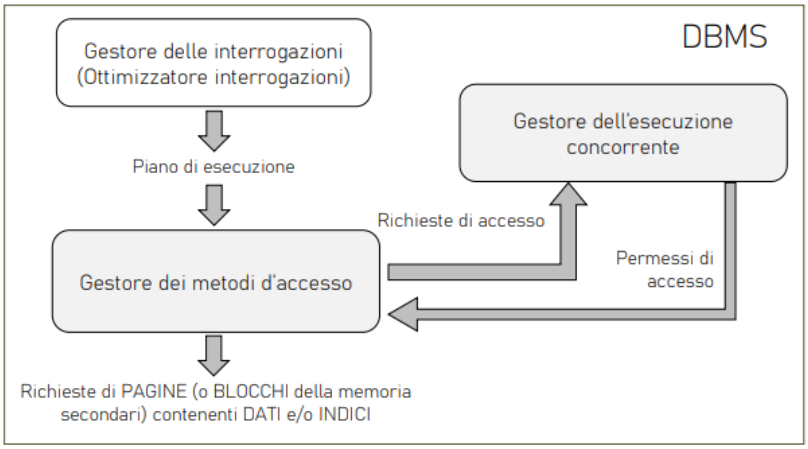


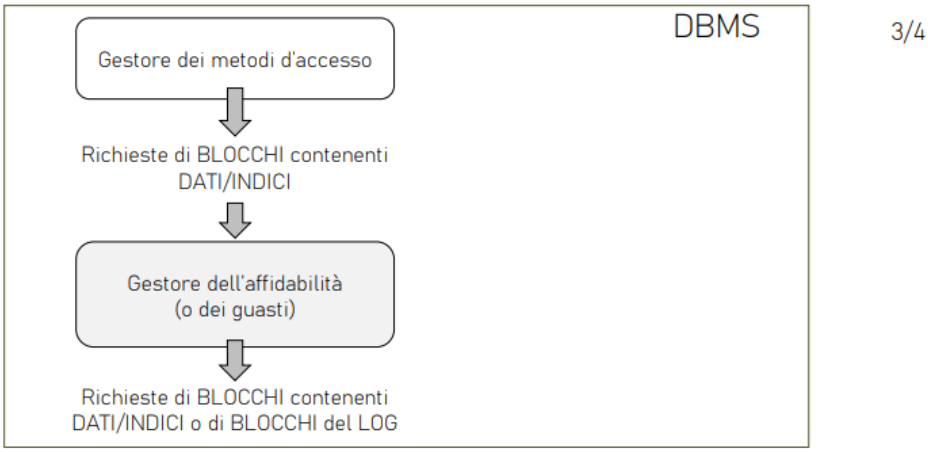

**Figura 1.4:** Architettura di riferimento di un DBMS.

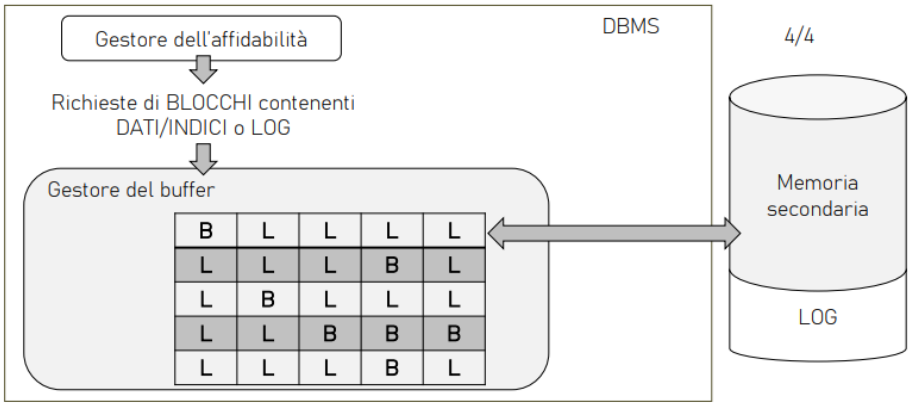



1/4




2/4




Quali modulo contribuiscono a garantire le proprietà delle transizioni?

**Gestore dei metodi d'accesso**: Consistenza
**Gestore dell'esecuzione concorrente**: Atomicità e Isolamento
**Gestore dell'affidabilità**: Atomicità e Persistenza

# 2

# SQL

Il **termine SQL** in origine era l'acronimo di **S**tructured **Q**uery **L**enguagge, ma ad oggi è diventato un nome proprio.

SQL è un linguaggio che permette di:

- **definire delle strutture dati e i vincoli di integrità**: Data Definition Lenguagge
- **manipolare i dati** (inserimento, aggiornamento e cancellazione): Data Manipilation Lenguagge
- **interrogare**: Data Query Lenguagge

Esistono diverse versioni:

| Nome informale | Nome ufficiale | Caratteristiche |
|---|---|---|
| SQL base | SQL-86 | Costrutti base |
| | SQL-89 | Integrità referenziale |
| SQL-2 | SQL-92 | Modello relazionale<br>Vari costrutti nuovi<br>3 livelli: entry, intermediate, full |
| SQL-3 | SQL:1999 | Modello relazionale a oggetti<br>Organizzato in diverse parti<br>Trigger, funzioni esterne, ... |
| | SQL:2003 | Estensioni del modello a oggetti<br>Eliminazione di costrutti non usati<br>Nuove parti: SQL/JRT, SQL/XML, ... |
| | SQL:2006 | Estensione della parte XML |
| | SQL:2008 | Lievi aggiunte (per esempio, `trigger instead of`) |
| | SQL:2011 | Ricco supporto per dati temporali |
| | SQL:2016 | Gestione del formato JSON |

La parte di SQL dedicata alle dichiarazioni fa parte dei DML.
Le dichiarazioni vengono espresse in modo dichiarativo: specifica l'obiettivo e non la procedura per ottenerlo (algebra relazionale).

## 2.1 Istruzione SELECT

```
SELECT [ DISTINCT ]
[ * | expression [[ AS ] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```

### 2.1.1 Struttura

**expression:** è un'espressione che determina un attributo.
**from_item:** è un'espressione che determina una sorgente per gli attributi.
**condition:** è un'espressione booleana per selezionare i valori degli attributi.
**grouping_element:** è un'espressione per poter eseguire operazioni su più valori di un attributo e considerare il risultato.

SELECT attributo
FROM tabella
[ WHERE Condizione ]

- SELECT, FROM e WHERE sono chiamate clausole
- Estrae dal prodotto cartesiano delle tabelle elencate nella clausola FROM, le righe che soddisfano la condizione specificata nella clausola WHERE.
  - Il risultato dell'interrogazione è una tabella con una riga per ogni riga prodotta dalla clausola FROM ed eventualmente filtrata dalla clausola WHERE (se esiste), le cui colonne dipendono dalla lista degli attributi nella clausola SELECT.

```
SELECT [ DISTINCT ]
[ * | expression [[ AS ] output_name ] [, ...] ]
```

- * è un'abbreviazione per indicare tutti gli attributi delle tabelle.
- expression è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).
- output_name è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione expression nella relazione risultato.
- DISTINCT: se presente richiede l'eliminazione delle tuple duplicate.

### 2.1.2 Interrogazione: Clausola SELECT

- Q1: Estrarre lo stipendio degli impiegati di cognome 'Rossi'

- SELECT Stipendio
  FROM Impiegato
  WHERE Cognome = 'Rossi';

Impiegato

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Bianchi | Produzione | 20 | 36 | Torino |
| Giovanni | Verdi | Amministrazione | 20 | 40 | Roma |
| Franco | Neri | Distribuzione | 16 | 45 | Napoli |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Lorenzo | Gialli | Direzione | 7 | 73 | Genova |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |
| Marco | Franco | Produzione | 20 | 46 | Roma |

| Stipendio |
|---|
| 45 |
| 80 |

SELECT Stipendio as Salario ← alias
FROM Impiegato
WHERE Cognome = 'Rossi'

Restituisce lo stesso risultato della query precedente, ma ridenomina (temporaneamente) la colonna output Stipendio

### 2.1.3 Interrogazione: Clausola SELECT ALL

SELECT Stipendio * ← Seleziona tutti gli attributi della tabella
FROM Impiegato
WHERE Cognome = 'Rossi'

L'asterisco permette di non elencare il nome di tutte le colonne di una tabella (simile alla proiezione)

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |

### 2.1.4 Interrogazione: Clasuola FROM

```
FROM from_item [, ...]
```

dove from_item può essere UNA delle seguenti clausole (versione semplificata):

1. table_name [[AS] alias [(column_alias [, ...])]
   - Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nella SELECT questi attributi sono identificati anteponendo il nome della tabella e un '.': nomeTabella.nomeAttributo
2. (other_select) [AS] nomeRisultato [(column_alias [,...])]

- è una SELECT innestata. Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.

3. from_item [NATURAL] join_type from_item [ON condition]
   - è la clausola JOIN.
   - Metodo più sofisticato di 1) che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. join_type specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti.

- Q2: Estrarre i nomi ed il cognome degli impiegati e le città in cui lavorano

- SELECT Impiegato.Nome, Impiegato.Cognome, Dipartimento.Città
  FROM Impiegato, Dipartimento
  WHERE Impiegato.Dipart = Dipartimento.Nome

Dipartimento

| Nome | Indirizzo | Città |
|---|---|---|
| Amministrazione | Via Tito Livio, 27 | Milano |
| Produzione | P.le Lavater, 3 | Torino |
| Distribuzione | Via Segre, 9 | Roma |
| Direzione | Via Tito Livio, 27 | Milano |
| Ricerca | Via Venosa, 6 | Milano |

| Impiegato.Nome | Impiegato.Cognome | Dipartimento.Città |
|---|---|---|
| Mario | Rossi | Milano |
| Carlo | Bianchi | Torino |
| Giovanni | Verdi | Milano |
| Franco | Neri | Roma |
| Carlo | Rossi | Milano |
| Lorenzo | Gialli | Milano |
| Paola | Rosati | Milano |
| Marco | Franco | Torino |

Impiegato

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Bianchi | Produzione | 20 | 36 | Torino |
| Giovanni | Verdi | Amministrazione | 20 | 40 | Roma |
| Franco | Neri | Distribuzione | 16 | 45 | Napoli |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Lorenzo | Gialli | Direzione | 7 | 73 | Genova |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |
| Marco | Franco | Produzione | 20 | 46 | Roma |

In alternatica nella voce FROM Impiegato, Dipartimento potevo utilizzare un alias.

### 2.1.5 Interrogazione: Clausola WHERE

```
WHERE condition
```

- La clausola WHERE ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori AND, OR e NOT
  - AND: seleziona le righe in cui tutti i predicati sono veri
  - OR: seleziona le righe in cui almeno un predicato è vero
  - NOT: seleziona le righe in cui il predicato è falso
- Ciascun predicato semplice usa gli operatori = , <>, <, >, <= e >= per confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante, oppure un'altra espressione.
- Per esprimere la precedenza degli operatori AND e OR si fa uso della parentesi.

- Q3: Estrarre i nomi ed il cognome degli impiegati che lavorano nell'ufficio 20 del dipartimento Amministrazione.

- SELECT Nome, Cognome
  FROM Impiegato
  WHERE Uffico = 20 AND
  Dipart = 'Amministrazione'

| Nome | Cognome |
|---|---|
| Giovanni | Verdi |

Dipartimento

| Nome | Indirizzo | Città |
|---|---|---|
| Amministrazione | Via Tito Livio, 27 | Milano |
| Produzione | P.le Lavater, 3 | Torino |
| Distribuzione | Via Segre, 9 | Roma |
| Direzione | Via Tito Livio, 27 | Milano |
| Ricerca | Via Venosa, 6 | Milano |

Impiegato

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Bianchi | Produzione | 20 | 36 | Torino |
| Giovanni | Verdi | Amministrazione | 20 | 40 | Roma |
| Franco | Neri | Distribuzione | 16 | 45 | Napoli |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Lorenzo | Gialli | Direzione | 7 | 73 | Genova |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |
| Marco | Franco | Produzione | 20 | 46 | Roma |

- Q4: Estrarre i nomi ed il cognome degli impiegati che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

- SELECT Nome, Cognome
  FROM Impiegato
  WHERE Dipart = 'Amministrazione'
  OR Dipart = 'Produzione'

| Nome | Cognome |
|---|---|
| Mario | Rossi |
| Carlo | Bianchi |
| Giovanni | Verdi |
| Paola | Rosati |
| Marco | Franco |

Dipartimento

| Nome | Indirizzo | Città |
|---|---|---|
| Amministrazione | Via Tito Livio, 27 | Milano |
| Produzione | P.le Lavater, 3 | Torino |
| Distribuzione | Via Segre, 9 | Roma |
| Direzione | Via Tito Livio, 27 | Milano |
| Ricerca | Via Venosa, 6 | Milano |

Impiegato

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Bianchi | Produzione | 20 | 36 | Torino |
| Giovanni | Verdi | Amministrazione | 20 | 40 | Roma |
| Franco | Neri | Distribuzione | 16 | 45 | Napoli |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Lorenzo | Gialli | Direzione | 7 | 73 | Genova |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |
| Marco | Franco | Produzione | 20 | 46 | Roma |

- Q5: Estrarre i nomi degli impiegati di cognome Rossi che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

- SELECT Nome
  FROM Impiegato
  WHERE Cognome = 'Rossi' AND
  (Dipart = 'Amministrazione' OR
  Dipart = 'Produzione')

| Nome |
|---|
| Mario |

Dipartimento

| Nome | Indirizzo | Città |
|---|---|---|
| Amministrazione | Via Tito Livio, 27 | Milano |
| Produzione | P.le Lavater, 3 | Torino |
| Distribuzione | Via Segre, 9 | Roma |
| Direzione | Via Tito Livio, 27 | Milano |
| Ricerca | Via Venosa, 6 | Milano |

Impiegato

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Bianchi | Produzione | 20 | 36 | Torino |
| Giovanni | Verdi | Amministrazione | 20 | 40 | Roma |
| Franco | Neri | Distribuzione | 16 | 45 | Napoli |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Lorenzo | Gialli | Direzione | 7 | 73 | Genova |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |
| Marco | Franco | Produzione | 20 | 46 | Roma |

## 2.2 Clausola WHERE

### 2.2.1 Operatore LIKE

- Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore LIKE che consente verificare se una stringa rispetta un certo "pattern" (oppure ILIKE per case insentive).
- Due caratteri speciali per il confronto "_" e "%":
  - "_": confronto con un carattere arbitrario
  - "%": confronto con una stringa di lunghezza arbitraria (anche 0) di caratteri arbitrari
- '_o%i' = stringa che ha una o in seconda posizione, seguita da una sequenza qualsiasi di caratteri (anche vuota) e che termina con una i.

- Q6: Estrarre gli impiegati che hanno un cognomen cha ha una "o" in seconda posizione e finisce con una "I".
- SELECT *
  FROM Impiegato
  WHERE Cognome LIKE '_o%i';

Dipartimento

| Nome | Indirizzo | Città |
|---|---|---|
| Amministrazione | Via Tito Livio, 27 | Milano |
| Produzione | P.le Lavater, 3 | Torino |
| Distribuzione | Via Segre, 9 | Roma |
| Direzione | Via Tito Livio, 27 | Milano |
| Ricerca | Via Venosa, 6 | Milano |

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |

Impiegato

| Nome | Cognome | Dipart | Ufficio | Stipendio | Città |
|---|---|---|---|---|---|
| Mario | Rossi | Amministrazione | 10 | 45 | Milano |
| Carlo | Bianchi | Produzione | 20 | 36 | Torino |
| Giovanni | Verdi | Amministrazione | 20 | 40 | Roma |
| Franco | Neri | Distribuzione | 16 | 45 | Napoli |
| Carlo | Rossi | Direzione | 14 | 80 | Milano |
| Lorenzo | Gialli | Direzione | 7 | 73 | Genova |
| Paola | Rosati | Amministrazione | 75 | 40 | Venezia |
| Marco | Franco | Produzione | 20 | 46 | Roma |

### 2.2.2 Operatore SIMILAR TO

- L'operatore SIMILAR TO è un LIKE più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).
- Esempi di componenti di espressioni regolari:
  - _ = 1 carattere qualsiasi.
  - % = 0 o più caratteri qualsiasi.
  - * = ripetizione del precedente match 0 o più volte.
  - + = ripetizione del precedente match UNA o più volte.
  - n,m = ripetizione del precedente match almeno n e non più di m volte.
  - ... = ... è un elenco di caratteri ammissibili

**Esempio**

Studenti con cognome che inizia con 'A' o 'B', o 'D', o 'N' e finisce con 'a':

```
SELECT cognome, nome, città FROM Studente
WHERE cognome SIMILAR TO '[ABDN]{1}%a';
```

| matricola | cognome | nome | indirizzo | città | media |
|---|---|---|---|---|---|
| IN0001 | Rossi | Marco | via nobile | Verona | 27.50 |
| IN0002 | Paoli | Paolo | via dritta | Padova | 28.6666 |
| IN0003 | Bianchi | Dante | via storta | VERONA | 18.345 |
| IN0004 | Rossi | Matteo | via nobile | Verona | 24.567 |
| IN0005 | Verdi | Luca | via nuova | Va | 28.6666 |
| IN0006 | Nocasa | Caio | | | |

### 2.2.3 Operatore BETWEEN

**Esempio**

Tutti gli studenti che hanno matricola tra 'IN0002' e 'IN0004'.

```
SELECT cognome, nome, matricola FROM Studente
WHERE matricola BETWEEN 'IN0002' AND 'IN0004';
```

| matricola | cognome | nome | indirizzo | città | media |
|---|---|---|---|---|---|
| IN0001 | Ros | | | na | 27.50 |
| IN0002 | Pa | | | va | 28.6666 |
| IN0003 | Bian | | | NA | 18.345 |
| IN0004 | Ros | | | na | 24.567 |
| IN0005 | Ver | | | | 28.6666 |
| IN0006 | Nocasa | Caio | | | |

| cognome | nome | matricola |
|---|---|---|
| Rossi | Marco | IN0001 |
| Paoli | Paolo | IN0002 |
| Bianchi | Dante | IN0003 |
| Rossi | Matteo | IN0004 |

### 2.2.4 Operatore IN

Nella clausola WHERE può apparire l'operatore [NOT] IN per testare l'appartenenza di un valore ad un insieme.

```
    WHERE attributo [NOT] IN (valore, [, ...]);
```

**Esempio**

Tutti gli studenti che hanno matricola nell'elenco 'IN0001', 'IN0003' e 'IN0005'.

```
SELECT cognome, nome, matricola FROM Studente
WHERE matricola IN('IN0001 ','IN0003 ','IN0005 ');
```

| matricola | cognome | nome | indirizzo | città | media |
|---|---|---|---|---|---|
| IN0001 | Rossi | Marco | via nobile | Verona | 27.50 |
| IN0002 | Paoli | Paolo | via dritta | Padova | 28.6666 |
| IN0003 | Bianchi | Dante | via storta | VERONA | 18.345 |
| IN0004 | Rossi | Matteo | via nobile | Verona | 24.567 |
| IN0005 | Verdi | Luca | via nuova | Va | 28.6666 |
| IN0006 | Nocasa | Caio | | | |

### 2.2.5 Operatore IS NULL

- Per verificare se un attributo ha un valore nullo o meno si utilizza il predicato IS NULL.
- Attributo IS NULL: ritorna true se l'attributo ha valore nullo.
- IS NOT NULL: è la negazione, ritorna true se l'attributo ha valore diverso da nullo.

```
    WHERE attributo IS [NOT] IN NULL;
```

SQL-2 implementa la logica a 3 valori:

- Un predicato semplice restituisce il valore NULL quando uno qualsiasi dei termini del predicato ha valore nullo.
- Il predicato IS NULL è una eccezione, restituendo sempre il valore vero oppure false, mai il valore NULL.

**Esempio**

Tutti gli studenti che NON hanno una città.

```
SELECT cognome, nome, città FROM Studente
WHERE città IS NULL;
```

| matricola | cognome | nome | indirizzo | città | media |
|---|---|---|---|---|---|
| IN0001 | Rossi | Marco | via nobile | Verona | 27.50 |
| IN0002 | Paoli | Paolo | via dritta | Padova | 28.6666 |
| IN0003 | Bianchi | Dante | via storta | VERONA | 18.345 |
| IN0004 | Rossi | Matteo | via nobile | Verona | 24.567 |
| IN0005 | Verdi | Luca | via nuova | Va | 28.6666 |
| IN0006 | Nocasa | Caio | | | |

## 2.3 SQL e ALGEBRA RELAZIONALE

Cerchiamo ora di capire la relazione con le corrispondenti interrogazioni procedurali in algebra relazionale.

Scrivere questa espressione in SQL:

SELECT $T_1$.attributo$_{1l}$,..., T$_h$.attributo$_{hm}$
FROM Tabella$_1$ as T$_1$,..., Tabella$_h$ as T$_h$
WHERE Condizione

Corrisponde a fare questa operazione in algebra relazionale:

$$\Pi_{T1.attributo1l, \ldots, Th.attributohm} (\sigma_{Condizione} (T1 \bowtie \ldots \bowtie Th))$$

### Esempi più concreti

Maternità

| Madre | Figlio |
|---|---|
| Luisa | Maria |
| Luisa | Luigi |
| Anna | Olga |
| Anna | Filippo |
| Maria | Andrea |
| Maria | Aldo |

Paternità

| Padre | Figlio |
|---|---|
| Sergio | Franco |
| Luigi | Olga |
| Luigi | Filippo |
| Franco | Andrea |
| Franco | Aldo |

Persone

| Nome | Età | Reddito |
|---|---|---|
| Andrea | 27 | 21 |
| Aldo | 25 | 15 |
| Maria | 55 | 42 |
| Anna | 50 | 35 |
| Filippo | 26 | 30 |
| Luigi | 50 | 40 |
| Franco | 60 | 20 |
| Olga | 30 | 41 |
| Sergio | 85 | 35 |
| Luisa | 75 | 87 |

Figura 2.1: Tabelle di riferimento

### 2.3.1 Selezione e proiezione

Nome e reddito delle persone con meno di trenta anni

**Algebra relazionale**: $\pi_{\text{Nome, Reddito}}(\sigma_{\text{Età}<30}(\text{Persone}))$

**SQL:**
```
SELECT Nome, Reddito
FROM Persone
WHERE età < 30
```

### SELECT: alias di tabella o di attributo

**Con ALIAS di Tabella**

```
SELECT p.nome, p.reddito
FROM persone as p
WHERE p.Età < 30
```

**ALIAS di attributo**

```
SELECT p.nome as nomePersona,
p.reddito as redditoPersona
FROM persone as p
WHERE p.Età < '30'
```

### 2.3.2 SELECT: Senza proiezione

Nome, reddito ed età delle persone con meno di trenta anni

**Algebra relazionale**: $\sigma_{\text{Età}<30}(\text{Persone})$

**SQL:**

```
SELECT *
FROM Persone
WHERE età < 30
```

### 2.3.3 Proiezione: gestione dei duplicati in SQL

Una significativa differenza tra SQL e algebra relazionale è la gestione dei duplicati. L'operazione di eliminazione delle righe doppie (ad esempio al seguito di una selezione con where) è molto costosa, quindi di default SQL mantiene le righe duplicate.

Il costrutto DISTINCT dopo la parola chiave SELECT permette di eliminare i duplicati. Il costrutto ALL permette invece di mantenerli, ma questa è già la scelta di default.

**Vediamone un esempio**

Impiegati

| Cognome | Nome | Filiale |
|---|---|---|
| Neri | Mario | Napoli |
| Neri | Giorgio | Napoli |
| Rossi | Claudio | Roma |

Trovare cognome e filiale di tutti gli impiegati

**Algebra relazionale**: $\pi_{\text{Cognome,Filiale}}(\text{Impiegati})$
Output:{(Neri, Napoli), (Rossi, Roma)}

Impiegati

| Cognome | Filiale |
|---|---|
| Neri | Napoli |
| Neri | Napoli |
| Rossi | Roma |

**SQL**
SELECT Cognome, Filiale
FROM Impiegati

Impiegati

| Cognome | Filiale |
|---|---|
| Neri | Napoli |
| Rossi | Roma |

**SQL**
SELECT DISTINCT Cognome, Filiale
FROM Impiegati

- Q7: Estrarre le città delle persone il cui cognome è Rossi

- SELECT Città
  FROM Persona
  WHERE Cognome = 'Rossi'

| Città |
|---|
| Verona |
| Verona |
| Milano |

- SELECT DISTINCT Città
  FROM Persona
  WHERE Cognome = 'Rossi'

| Città |
|---|
| Verona |
| Milano |

Persona

| CodFiscale | Nome | Cognome | Città |
|---|---|---|---|
| RSSMRA55B21T234J | Mario | Rossi | Verona |
| BNCCLR69T30H745Z | Carlo | Bianchi | Roma |
| RSSGNN41A31B344C | Giovanni | Rossi | Verona |
| RSSPRT75C12F205V | Pietro | Rossi | Milano |

### 2.3.4 SELECT: Selezione, Proiezione, Join

L'istruzione SELECT, con una sola relazione nella clausola FROM permette di realizzare: selezioni, proiezioni, ridenominazioni.
Invece con più relazioni nella clausola FROM si realizzano join (e prodotti cartesiani)

| | |
|---|---|
| SELECT | $\pi$Proiezione + $\rho$Ridenominazione (con AS) |
| FROM | $\bowtie$Join(prodotto cartesiano) |
| WHERE | $\sigma$Selezione + $\bowtie$ Join(condizion) |

### 2.3.5 Interrogazioni con JOIN

In SQL il join si può definire in due modi diversi

- Il primo non richiede l'uso di nuova sintassi, e specifica il legame tra le tabelle coinvolte a livello della clausola WHERE.

- Il secondo usa una sintassi riservata a livello di clausola FROM (useremo questo).

La clausola FROM ammette come argomento:

table_name [[AS] name]] [, ...]

Si è visto che se sono presenti due o più nomi di tabelle, si esegue il prodotto cartesiano tra tutte le tabelle e lo schema del risultato può contenere tutti gli attributi del prodotto cartesiano.
Il prodotto cartesiano di due o più tabelle è un CROSS JOIN.
A partire da SQL-2, esistono altri tipi di JOIN (join_type): INNER JOIN, LEFT [OUTER] JOIN, RIGHT [OUTER] JOIN e FULL [OUTER] JOIN.

### Sintassi generale

```
table_name [NATURAL] join_type table_name [ ON join_condition
    [, ...]].
```

dove join_condition è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola WHERE

**Prodotto Cartesiano**

- Formulazione con JOIN:

```
SELECT I. cognome , R. nomeRep
FROM Impiegato AS I
    CROSS JOIN Reparto AS R;
```

- Formulazione con ‘,’:

```
SELECT I. cognome , R. nomeRep
FROM Impiegato AS I, Reparto AS R;
```

| cognome | nome |
|---|---|
| Rossi | Acquisti |
| Verdi | Acquisti |
| Rossi | Vendite |
| Verdi | Vendite |

#### 2.3.5.1 Interrogazioni con INNER JOIN

### Sintassi generale

```
table1 [ INNER ] JOIN table2 ON join_condition [, ...]
```

Rappresenta il tradizionale $\theta$ join dell'algebra relazionale. Combina ciascuna riga r1 di table1 con ciascuna riga di table2 che soddisfa la condizione della clausola ON

**SQL**

```
SELECT I.cognome , R.nomeRep,
R.sede
FROM Impiegato AS I
  INNER JOIN Reparto AS R
  ON I.nomerep = R.nomerep;
```

| cognome | nomerep | sede |
|---|---|---|
| Rossi | Acquisti | Verona |
| Verdi | Vendite | Verona |

### 2.3.5.2 Interrogazioni con LEFT OUTER JOIN

**Sintassi generale**

```
table1 LEFT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r1 di table1 che non soddisfa la condizione con qualsiasi riga di table2, si aggiunge una riga al risultato con i valori di r1 e assegnando NULL agli altri attributi..

**SQL**

```
INSERT INTO Reparto (nomerep, sede, telefono)
VALUES ('Finanza', 'Padova', '02 8028888');
SELECT R.nomeRep, I.cognome
FROM Reparto AS R
  LEFT OUTER JOIN Impiegato AS I
  ON I.nomerep = R.nomerep;
```

| nomerep | cognome |
|---|---|
| Acquisti | Rossi |
| Vendite | Verdi |
| Finanza | |